

理论计算机科学导引

电子化讲义

Huijiao

2026 年 8 月 4 日

目录

第 1 讲 引子	1
1.1 二进制串与编码	1
1.2 可数集合	3
1.3 问题的公理化定义	4
第 2 讲 布尔电路	5
2.1 电路模型	5
2.2 AON-Circ 程序与 NAND 门	5
2.3 任意布尔函数的电路实现	6
2.3.1 NAND-Circ 程序中的语法糖 (syntactic sugar)	7
2.3.2 ADD 与 LOOKUP	7

第 1 讲 引子

理论计算机科学经常把两个层面分开讨论：函数 (function) 描述要计算什么，是问题的规格；程序 (program) 描述如何计算，是规格的一种实现。从函数到程序，就是把抽象的东西具体化。本讲先用二进制串 (binary string) 形式化“描述”，再比较函数与程序在数量上的差异。

全文约定

$$\mathbb{N} = \{0, 1, 2, \dots\}, \quad \Sigma = \{0, 1\}.$$

1.1 二进制串与编码

定义 1.1 (二进制串). 对任意 $n \in \mathbb{N}$, 定义

$$\Sigma^n = \{(a_1, \dots, a_n) : a_i \in \Sigma\}.$$

特别地, $\Sigma^0 = \{\varepsilon\}$, 其中 ε 表示空串 (empty string)。记

$$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n, \quad \Sigma^+ = \Sigma^* \setminus \{\varepsilon\}.$$

对 $x \in \Sigma^n$, 记其长度为 $|x| = n$ 。

对 $x, y \in \Sigma^*$, 把二者的串联记为 xy 。如果存在 $z \in \Sigma^*$ 使得 $y = xz$, 则称 x 是 y 的前缀 (prefix)。如果还能取到 $z \neq \varepsilon$, 则称 x 是 y 的真前缀 (proper prefix)。

定义 1.2 (编码). 集合 A 上的一个 (二进制) 编码 (encoding) 是单射 (injective map)

$$E : A \longrightarrow \Sigma^*.$$

这里, 单射是指: 对任意 $x, y \in A$, 若 $E(x) = E(y)$, 则 $x = y$ 。

例 1.1. 令 $b(n)$ 为自然数 n 的标准二进制表示, 并尝试用

$$P(a, b) = b(a)b(b)$$

编码 $\mathbb{N} \times \mathbb{N}$ 。这个映射不是单射。例如

$$P(1, 6) = 1\ 110 = 1110 = 11\ 10 = P(3, 2).$$

问题在于直接串联后无法确定两个分量的分界位置。因此我们要考虑一种更优秀的编码, 来避免这个问题。

定义 1.3 (无前缀编码). 编码 $E: A \rightarrow \Sigma^+$ 称为无前缀的 (prefix-free), 如果对任意不同的 $x, y \in A$, 码字 $E(x)$ 都不是 $E(y)$ 的前缀。

记 A^* 为由 A 中元素组成的所有有限序列的集合, 其空序列仍记为 ε 。对映射 $E: A \rightarrow \Sigma^*$, 定义其逐项串联扩张

$$\overline{E}(a_1, \dots, a_n) = \begin{cases} E(a_1) \cdots E(a_n), & n > 0, \\ \varepsilon, & n = 0. \end{cases} \quad (1.1)$$

这个定义很自然, 我们要说明这个自然定义确实是个单射。

引理 1.1. 若 $E: A \rightarrow \Sigma^+$ 是无前缀编码, 则由 式 (1.1) 定义的 $\overline{E}: A^* \rightarrow \Sigma^*$ 是单射。

证明. 假设两组序列的编码相同:

$$\overline{E}(a_1, \dots, a_p) = \overline{E}(b_1, \dots, b_q).$$

码字非空, 所以 $p = 0 \iff q = 0$ 。若 $p, q > 0$, 则 $E(a_1)$ 与 $E(b_1)$ 中较短的码字一定是较长码字的前缀。由无前缀性可知

$$E(a_1) = E(b_1) \implies a_1 = b_1.$$

两边消去首个码字并归纳, 得到

$$p = q, \quad a_i = b_i \quad (1 \leq i \leq p).$$

故 $\overline{E}$ 是单射。 □

定理 1.1 (编码的无前缀化). 若存在编码 $E: A \rightarrow \Sigma^*$, 则存在无前缀编码 $E_{\text{pf}}: A \rightarrow \Sigma^+$, 并且对每个 $a \in A$ 都有

$$|E_{\text{pf}}(a)| = 2|E(a)| + 2.$$

证明. 定义 $h(0) = 00$ 、 $h(1) = 11$, 并令 h 逐位作用于二进制串。取

$$E_{\text{pf}}(a) = h(E(a))01, \quad |E_{\text{pf}}(a)| = 2|E(a)| + 2.$$

按两个比特分组时, 正文块属于 $\{00, 11\}$, 终止块为 01 。若 $E_{\text{pf}}(a)$ 是 $E_{\text{pf}}(b)$ 的前缀, 则短串的终止块必须与长串的某一块对齐。正文中没有 01 , 故它只能与长串的终止块对齐。于是

$$E_{\text{pf}}(a) = E_{\text{pf}}(b) \implies E(a) = E(b) \implies a = b.$$

因此 E_{pf} 无前缀。 □

思考题. 能否把长度开销改进到 $|E(a)| + O(\log|E(a)|)$?

解答. 可以. 令

$$n = |E(a)|, \quad \ell = |\text{bin}(n+1)|,$$

并定义

$$L(n) = 1^\ell 0 \text{ bin}(n+1), \quad E'(a) = L(n)E(a).$$

解码过程为

$$1^\ell 0 \longrightarrow \ell \longrightarrow \text{bin}(n+1) \longrightarrow n \longrightarrow E(a).$$

其中最后一步恰好读取 n 位. 并且

$$|E'(a)| = n + 2\lfloor \log_2(n+1) \rfloor + 3 = n + O(\log(n+1)).$$

$L(n)$ 自定界; 当 n 相同时, $E'(a)$ 等长. 因此该构造无前缀. $\square$

推论 1.1. 若存在编码 $E: A \rightarrow \Sigma^*$, 则存在编码 $F: A^* \rightarrow \Sigma^*$.

证明. 先用定理 1.1 把 E 无前缀化, 再用引理 1.1 逐项串联即可. $\square$

1.2 可数集合

定义 1.4 (至多可数). 如果存在单射 $f: A \rightarrow \mathbb{N}$, 则称集合 A 是至多可数的 (at most countable). 换言之, A 或者是有限集, 或者可以与自然数集 $\mathbb{N}$ 建立双射 (bijection). 当 $A \neq \emptyset$ 时, 这也等价于存在满射 (surjection) $g: \mathbb{N} \rightarrow A$.

注. 非空条件只用于满射表述, 因为不存在 $\mathbb{N} \rightarrow \emptyset$. 有些文献把本文的“至多可数”简称为“可数”.

引理 1.2. Σ^* 是可数无限集.

证明. 对 $x \in \Sigma^n$, 令 $\text{val}_2(x)$ 为其二进制数值 (允许前导零), 并定义

$$\nu(x) = 2^n - 1 + \text{val}_2(x).$$

于是

$$\nu(\Sigma^n) = \{2^n - 1, \dots, 2^{n+1} - 2\}, \quad \mathbb{N} = \bigsqcup_{n \geq 0} \nu(\Sigma^n).$$

故 $\nu: \Sigma^* \rightarrow \mathbb{N}$ 是双射. $\square$

定理 1.2 (可编码性与可数性). 集合 A 至多可数, 当且仅当存在编码 $E: A \rightarrow \Sigma^*$.

证明. 由引理 1.2, 存在双射 $\nu: \Sigma^* \rightarrow \mathbb{N}$.

若 $E: A \rightarrow \Sigma^*$ 是单射, 则

$$\nu \circ E: A \rightarrow \mathbb{N}$$

也是单射. 反之, 若 $f: A \rightarrow \mathbb{N}$ 是单射, 令 $c(n) = 1^n 0$, 则

$$c \circ f: A \rightarrow \Sigma^*$$

是一个编码. $\square$

1.3 问题的公理化定义

因为我们已经有编码了，所以我们可以把问题公理化了。

定义 1.5 (问题). 在本讲义中，一个 (计算) 问题 (computational problem) 是函数

$$f : \Sigma^* \longrightarrow \Sigma^*.$$

所有问题构成的集合记为

$$\mathcal{P} = (\Sigma^*)^{\Sigma^*}.$$

定理 1.3. 问题集合 $\mathcal{P}$ 不可数。

证明 (*Cantor* 对角化). 由[引理 1.2](#), 可写

$$\Sigma^* = \{s_0, s_1, s_2, \dots\}.$$

反设 $\mathcal{P} = \{f_0, f_1, f_2, \dots\}$ 。定义

$$g(s_i) = \begin{cases} 0, & f_i(s_i) \neq 0, \\ 1, & f_i(s_i) = 0. \end{cases}$$

因而

$$\forall i \in \mathbb{N}, \quad g(s_i) \neq f_i(s_i) \implies g \neq f_i.$$

于是 $g \in \mathcal{P}$ 却不在上述枚举中，矛盾。

□

第 2 讲 布尔电路

布尔电路 (Boolean circuit) 用有限个逻辑门 (logic gate) 把输入比特 (bit) 变成输出比特。本讲使用三种基本逻辑门：与门 (AND)、或门 (OR) 和非门 (NOT)。

2.1 电路模型

一个布尔电路 C 可以看成有限的有向无环图 (directed acyclic graph, DAG)。图中包含：

1. n 个输入结点 $x_0, \dots, x_{n-1}$;
2. 若干内部结点, 每个结点标记为 AND、OR 或 NOT 门;
3. m 个指定的输出结点 $y_0, \dots, y_{m-1}$ 。

AND 和 OR 门各有两个输入, NOT 门有一个输入。因为图中没有有向环, 所以各个门可以按拓扑顺序 (topological order) 依次求值。电路的规模 (size) 是其中逻辑门的数量, 记为 $|C|$ 。

定义 2.1 (电路计算). 设 $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ 。如果对每个输入 $x \in \{0, 1\}^n$, 电路 C 的 m 个输出比特恰好组成 $f(x)$, 则称 C 计算函数 f 。

例 2.1. 异或函数 (exclusive OR, XOR) 可以写成

$$\text{XOR}(a, b) = \neg(a \wedge b) \wedge (a \vee b),$$

因而可以用 AND、OR 和 NOT 门组成的布尔电路计算。

2.2 AON-Circ 程序与 NAND 门

定义 2.2 (AON-Circ 程序). AON-Circ 程序是一列按顺序执行的赋值语句 (assignment statement)。每条语句调用一次 AND、OR 或 NOT, 并且只能读取输入变量或此前语句已经算出的变量。程序最后指定 m 个变量作为输出。程序的规模是赋值语句的行数。

例如，一条语句可以具有以下三种形式之一：

$$z_i \leftarrow z_j \wedge z_k, \quad z_i \leftarrow z_j \vee z_k, \quad z_i \leftarrow \neg z_j,$$

其中右侧的变量必须已经定义。

定理 2.1. 函数 f 能由布尔电路计算，当且仅当它能由 *AON-Circ* 程序计算。两种表示可以保持规模不变：电路的门数等于对应程序的行数。

证明. 给定电路，按拓扑顺序为每个门写一条赋值语句，即得到 *AON-Circ* 程序。反过来，为程序的每一行建立一个门，并把该行读取的变量连接到这个门，即得到布尔电路。每个门与每行语句一一对应。□

与非门 (NAND) 定义为

$$\text{NAND}(a, b) = \neg(a \wedge b).$$

只使用 NAND 门便可实现三个基本门：

$$\begin{aligned} \neg a &= \text{NAND}(a, a), \\ a \wedge b &= \text{NAND}(\text{NAND}(a, b), \text{NAND}(a, b)), \\ a \vee b &= \text{NAND}(\text{NAND}(a, a), \text{NAND}(b, b)). \end{aligned}$$

因此 NAND 电路与普通布尔电路可以相互转换。若原电路规模为 s ，则转换后分别满足

$$|C_{\text{AON}}| \leq 2|C_{\text{NAND}}|, \quad |C_{\text{NAND}}| \leq 3|C_{\text{AON}}|.$$

2.3 任意布尔函数的电路实现

定理 2.2. 对任意 $n, m \geq 1$ 以及任意函数

$$f : \{0, 1\}^n \longrightarrow \{0, 1\}^m,$$

都存在计算 f 的布尔电路。进一步，电路规模可以达到

$$|C| = O\left(\frac{m2^n}{n}\right).$$

粗略上界的构造. 先考虑 f 的第 j 个输出比特 f_j 。对每个 $a = (a_0, \dots, a_{n-1}) \in \{0, 1\}^n$ ，定义

$$T_a(x) = \bigwedge_{r=0}^{n-1} \ell_{a,r}(x_r), \quad \ell_{a,r}(x_r) = \begin{cases} x_r, & a_r = 1, \\ \neg x_r, & a_r = 0. \end{cases}$$

当且仅当 $x = a$ 时, $T_a(x) = 1$ 。因此

$$f_j(x) = \bigvee_{\substack{a \in \{0,1\}^n \\ f_j(a)=1}} T_a(x).$$

这就是析取范式 (disjunctive normal form, DNF) 的直接构造。每个 T_a 使用 $O(n)$ 个门, 而这样的项至多有 2^n 个, 所以一个输出比特需要 $O(n2^n)$ 个门。分别构造 m 个输出, 便得到

$$|C| = O(mn2^n).$$

定理 2.2 中的 $O(m2^n/n)$ 上界去掉了这个直接构造中的冗余。这个更精细的上界将在后文证明。

定义 2.3 (通用门组). 如果只使用门集合 $\mathcal{F}$ 中的门就能计算任意布尔函数, 则称 $\mathcal{F}$ 是通用的 (universal)。

由定理 2.2 和上面的三个恒等式, NAND 门本身就是通用的。因此, 要判断一组门 $\mathcal{F}$ 是否通用, 只需判断能否仅用 $\mathcal{F}$ 中的门实现 NAND 函数。

为了证明定理 2.2, 我们先来介绍一些特性和例子。

2.3.1 NAND-Circ 程序中的语法糖 (syntactic sugar)

为了让程序更容易书写, 可以暂时加入以下三种语法糖:

1. **定长循环 (fixed-length loop)**: 把循环体复制固定次数;
2. **用户定义过程 (user-defined procedure)**: 在调用处展开过程体;
3. **条件语句 (conditional statement)**: 同时计算两个分支, 再用选择器 (multiplexer, MUX) 输出其中一个。

条件语句中的一比特选择器可以写成

$$\text{MUX}(b, u, v) = (\neg b \wedge u) \vee (b \wedge v),$$

其中 $b = 0$ 时输出 u , $b = 1$ 时输出 v 。循环次数和过程参数一旦固定, 上述写法都能展开成普通的 NAND-Circ 程序, 因此不会增强计算能力。

2.3.2 ADD 与 LOOKUP

例 2.2 (ADD). 输入包含两个 n 位二进制数。按从低位到高位顺序, 将它们分别记为

$$(x_0, \dots, x_{n-1}), \quad (x_n, \dots, x_{2n-1})$$

加法函数为

$$\text{ADD}_n : \{0, 1\}^{2n} \longrightarrow \{0, 1\}^{n+1}.$$

令 $\text{MAJ}(a, b, c)$ 表示三个比特的多数值 (majority), 即

$$\text{MAJ}(a, b, c) = (a \wedge b) \vee (a \wedge c) \vee (b \wedge c).$$

可以用如下定长循环实现逐位进位加法:

```
result = [0] * (n + 1)
carry = [0] * (n + 1)
for i in range(n):
    result[i] = XOR(carry[i], XOR(x[i], x[i+n]))
    carry[i+1] = MAJ(carry[i], x[i], x[i+n])
result[n] = carry[n]
return result
```

XOR 和 MAJ 都能用常数个 NAND 门实现。把循环展开后, 程序共有 $O(n)$ 行, 因而加法电路的规模为 $O(n)$ 。

例 2.3 (LOOKUP). 输入由 2^k 个数据比特 $x_0, \dots, x_{2^k-1}$ 和 k 个索引比特 $i_0, \dots, i_{k-1}$ 组成。令

$$j = \sum_{r=0}^{k-1} i_r 2^{k-1-r}.$$

查表函数 (lookup function) 返回第 j 个数据比特:

$$\text{LOOKUP}_k : \{0, 1\}^{2^k+k} \longrightarrow \{0, 1\}, \quad \text{LOOKUP}_k(x, i) = x_j.$$

它可以递归地把数据表分成两半:

```
lookup(x, index):
    if len(index) == 1:
        if index[0] == 0: return x[0]
        else: return x[1]
    half = len(x) / 2
    if index[0] == 0:
        return lookup(x[0:half], index[1:])
    else:
        return lookup(x[half:], index[1:])
```

对电路而言, 条件语句的两个分支都要构造, 再由 MUX 选择输出。若 $L(k)$ 表示展开后的程序行数, 则

$$L(k) = 2L(k-1) + O(1), \quad L(1) = O(1).$$

因此 $L(k) = O(2^k)$, LOOKUP 电路的规模也是 $O(2^k)$ 。